lulipro - 代码钢琴家

学习知识的最大价值在于愉悦自己的大脑，其次才是让自己生存。

树莓派wiringPi库详解

wiringPi是一个很棒的树莓派IO控制库，使用C语言开发，提供了丰富的接口：GPIO控制，中断，多线程，等等。java 的pi4j项目也是基于wiringPi的，我最近也在看源代码，到时候整理好了会放出来的。

下面开始wiringPi之旅吧！

安装

进入 wiringPi的github (https://git.drogon.net/?p=wiringPi;a=summary) 下载安装包。点击页面的第一个链接的右边的snapshot,下载安装压缩包。

然后进入安装包所在的目录执行以下命令：

```
>tar xfz wiringPi-98bcb20.tar.gz   //98bcb20为版本标号，可能不同
>cd wiringPi-98bcb20
>./build
```

验证wiringPi的是否安装成功，输入gpio -v，会在终端中输出相关wiringPi的信息。否则安装失败。

编译 和运行

假如你写了一个LEDtest.c 的项目，则如下。

```
编译：

g++ -Wall -o LEDtest LEDtest.cpp  -lwiringPi         //使用C++编程 ， -Wall 是为了使能所有警告，以便发现程序中的问

gcc -Wall -o LEDtest LEDtest.c   -lwiringPi          //使用C语言编程


运行：

sudo ./LEDtest
```

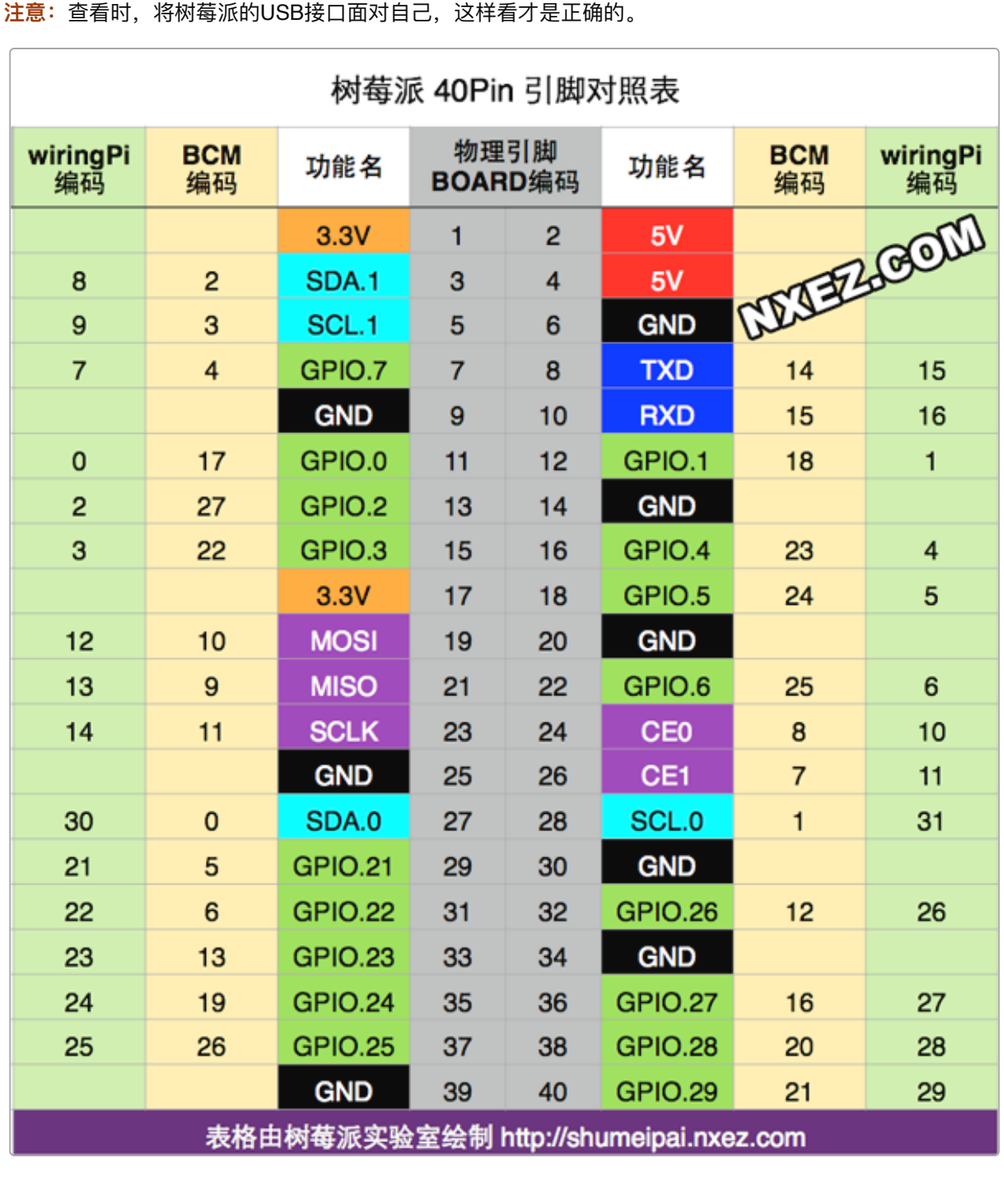
查看引脚编号表格

使用如下控制台下命令

```
> gpio readall
```

也可以查看下面的图。

注意：查看时，将树莓派的USB接口面对自己，这样看才是正确的。



树莓派 40Pin 引脚对照表

wiringPi编码	BCM编码	功能名	物理引脚 BOARD编码		功能名	BCM编码	wiringPi编码
		3.3V	1	2	5V		
8	2	SDA.1	3	4	5V		
9	3	SCL.1	5	6	GND		
7	4	GPIO.7	7	8	TXD	14	15
		GND	9	10	RXD	15	16
0	17	GPIO.0	11	12	GPIO.1	18	1
2	27	GPIO.2	13	14	GND		
3	22	GPIO.3	15	16	GPIO.4	23	4
		3.3V	17	18	GPIO.5	24	5
12	10	MOSI	19	20	GND		
13	9	MISO	21	22	GPIO.6	25	6
14	11	SCLK	23	24	CE0	8	10
		GND	25	26	CE1	7	11
30	0	SDA.0	27	28	SCL.0	1	31
21	5	GPIO.21	29	30	GND		
22	6	GPIO.22	31	32	GPIO.26	12	26
23	13	GPIO.23	33	34	GND		
24	19	GPIO.24	35	36	GPIO.27	16	27
25	26	GPIO.25	37	38	GPIO.28	20	28
		GND	39	40	GPIO.29	21	29

表格由树莓派实验室绘制 http://shumeipai.nxez.com

wiringPi库API大全

在使用wiringPi库时，你需要包含头文件 #include<wiringPi.h>。凡是写wiringPi的程序，都包含这个头文件。

硬件初始化函数

使用wiringPi时，你必须在执行任何操作前初始化树莓派，否则程序不能正常工作。

可以调用下表函数之一进行初始化，它们都会返回一个int ， 返回 -1 表示初始化失败。

int wiringPiSetup (void)	返回:执行状态，-1表示失败	当使用这个函数初始化树莓派引脚时，程序使用的是wiringPi 引脚编号表。引脚的编号为 0~16 需要root权限
int wiringPiSetupGpio (void)	返回执行状态，-1表示失败	当使用这个函数初始化树莓派引脚时，程序中使用的是BCM GPIO 引脚编号表。 需要root权限
wiringPiSetupPhys(void)	不常用，不做介绍	/
wiringPiSetupSys (void) ;	不常用，不做介绍	/

通用GPIO控制函数

void pinMode (int pin, int mode)	pin：配置的引脚 mode:指定引脚的IO模式 可取的值：INPUT、OUTPUT、PWM_OUTPUT，GPIO_CLOCK	作用：配置引脚的IO模式 注意： 只有wiringPi 引脚编号下的1脚（BCM下的18脚） 支持PWM输出 只有wiringPi编号下的7（BCM下的4号）支持GPIO_CLOCK输出
void digitalWrite (int pin, int value)	pin：控制的引脚 value：引脚输出的电平值。 可取的值：HIGH，LOW分别代表高低电平	让对一个已近配置为输出模式的 引脚 输出指定的电平信号
int digitalRead (int pin)	pin：读取的引脚 返回：引脚上的电平，可以是LOW HIGH 之一	读取一个引脚的电平值 LOW HIGH ，返回
void analogWrite(int pin, int value)	pin:引脚 value：输出的模拟量	模拟量输出 树莓派的引脚本身是不支持AD转换的，也就是不能使用模拟量的API， 需要增加另外的模块
int analogRead (int pin)	pin：引脚 返回：引脚上读取的模拟量	模拟量输入 树莓派的引脚本身是不支持AD转换的，也就是不能使用模拟量的API， 需要增加另外的模块
void pwmWrite (int pin, int value)	pin：引脚 value：写入到PWM寄存器的值，范围在0~1024之间。	输出一个值到PWM寄存器，控制PWM输出。 pin只能是wiringPi 引脚编号下的1脚（BCM下的18脚）
void pullUpDnControl (int pin, int pud)	pin：引脚 pud：拉电阻模式 可取的值：PUD_OFF 不启用任何拉电阻。关闭拉电阻。 PUD_DOWN 启用下拉电阻，引脚电平拉到GND PUD_UP 启用上拉电阻，引脚电平拉到3.3v	对一个设置IO模式为 INPUT 的输入引脚设置拉电阻模式。 与Arduino不同的是，树莓派支持的拉电阻模式更丰富。 树莓派内部的拉电阻达50K欧姆

LED闪烁程序

View Code

PWM输出控制LED呼吸灯的例子

View Code

时间控制函数

unsigned int millis (void)	这个函数返回 一个 从你的程序执行 wiringPiSetup 初始化函数（或者wiringPiSetupGpio ） 到 当前时间 经过的 毫秒数。 返回类型是unsigned int，最大可记录 大约49天的毫秒时长。
unsigned int micros (void)	这个函数返回 一个 从你的程序执行 wiringPiSetup 初始化函数（或者wiringPiSetupGpio ） 到 当前时间 经过的 微秒数。 返回类型是unsigned int，最大可记录 大约71分钟的时长。
void delay (unsigned int howLong)	将当前执行流暂停 指定的毫秒数。因为Linux本身是多线程的，所以实际暂停时间可能会长一些。参数是unsigned int 类型，最大延时时间可达49天
void delayMicroseconds (unsigned int howLong)	将执行流暂停 指定的微秒数（1000微秒 = 1毫秒 = 0.001秒）。 因为Linux本身是多线程的，所以实际暂停时间可能会长一些。参数是unsigned int 类型，最大延时时间可达71分钟

中断

wiringPi提供了一个中断处理注册函数，它只是一个注册函数，并不处理中断。他无需root权限。

int wiringPiISR (int pin, int edgeType, void (*function)(void))	返回值：返回负数则代表注册失败 pin：接受中断信号的引脚 edgeType：触发的方式。 INT_EDGE_FALLING：下降沿触发 INT_EDGE_RISING：上升沿触发 INT_EDGE_BOTH ：上下降都会触发 INT_EDGE_SETUP：编程时用不到。 function：中断处理函数的指针，它是一个无返回值，无参数的函数。	注册的函数会在中断发生时执行 和51单片机不同的是：这个注册的中断处理函数会和main函数并发执行（同时执行，谁也不耽误谁） 当本次中断函数还未执行完毕，这个时候树莓派又触发了一个中断，那么这个后来的中断不会被丢弃，它仍然可以被执行。但是wiringPi最多可以跟踪并记录后来的仅仅1个中断，如果不止1个，则他们会被忽略，得不到执行。

通过1脚检测 因为按键按下引发的 下降沿，触发中断，反转11控制的LED

```
#include<iostream>
#include<wiringPi.h>
#include<cstdlib>
using namespace std;

void ButtonPressed(void);
void setup();

/********************************/
const int LEDPin = 11;
const int ButtonPin = 1;
/*******************************/

int main()
{

    setup();

    //注册中断处理函数
    if(0>wiringPiISR(ButtonPin,INT_EDGE_FALLING,ButtonPressed))
    {
        cerr<<"interrupt function register failure"<<endl;
        exit(-1);
    }


    while(1)
    ;

    return 0;
}

void setup()
{
    if(-1==wiringPiSetup())
    {
        cerr<<"wiringPi setup error"<<endl;
        exit(-1);
    }

    pinMode(LEDPin,OUTPUT);     //配置11脚为控制LED的输出模式
    digitalWrite(LEDPin,LOW);  //初始化为低电平

    pinMode(ButtonPin,INPUT);            //配置1脚为输入
    pullUpDnControl(ButtonPin,PUD_UP);  //将1脚上拉到3.3v

}


//中断处理函数：反转LED的电平
void ButtonPressed(void)
{

    digitalWrite(LEDPin,  (HIGH==digitalRead(LEDPin))?LOW:HIGH );

}
```

多线程

wiringPi提供了简单的Linux系统下的通用的 Posix threads线程库接口来支持并发。

int piThreadCreate(name)	name:被包装的线程执行函数
	返回：状态码。返回0表示成功启动，反之失败。
	源代码：
	int piThreadCreate (void *(*fn)(void *))
	{
	pthread_t myThread ;
	return pthread_create (&myThread, NULL, fn, NULL) ;
	}
		包装一个用PI_THEEAD定义的函数为一个线程，并启动这个线程。
		首先你需要通过以下方式创建一个特特殊的函数，这个函数中的代码就是在新的线程中将执行的代码。，myTread是你自己线程的名字，可自定义。
		PI_THREAD (myThread)
		{
		//在这里面写上的代码会和主线程并发执行。
		}
		在wiringPi.h中，我发现这样一个宏定义：#define PI_THREAD(X) void *X (void *dummy)
		那么，被预处理后我们写的线程函数会变成下面这个样子，请注意返回值，难怪我每次写都会警告，因为没有返回一个指针，
		那么，以后注意返回NULL，或者 (void*)0
		void *myThread (void *dummy)
		{
		//在这里面写上的代码会和主线程并发执行。
		}
		//在这里面写上的代码会和主线程并发执行。

piLock(int keyNum)	keyNum:0-3的值，每一个值代表一把锁	使能同步锁。wiringPi只提供了4把锁，也就是keyNum只能取0~3的值，官方认为有这4把锁就够了。
		keyNum：0,1,2,3 每一个数字就代表一把锁。
		源代码：
		void piLock (int keyNum)
		{
		pthread_mutex_lock (&piMutexes [keyNum]) ;
		}

piUnlock(int keyNum)	keyNum:0-3的值，每一个值代表一把锁	解锁，或者说让出锁。
		源代码：
		void piUnlock (int key)
		{
		pthread_mutex_unlock (&piMutexes [key]) ;
		}

int piHiPri (int priority)	priority：优先级指数，0~99 返回值：0，成功 -1:，失败	设定线程的优先级，设定线程的优先级变高，不会使程序运行加快，但会使这个线程获得相当更多的时间片。priority是相对的。比如你的程序只用到了主线程， 和另一个线程A，主线程设定优先级为1，A线程设定为2，那也代表A比main线程优先级高。

凡是涉及到多线程编程，就会涉及到线程安全的问题，多线程访问同一个数据，需要使用同步锁来保障数据操作正确性和符合预期。

当A线程锁上 锁S 后，其他共用这个锁的竞争线程，只能等到锁被释放，才能继续执行。

成功执行了piLock 函数的线程将拥有这把锁。其他线程想要拥有这把锁必须等到这个线程释放锁，也就是这个线程执行piUnlock后。

同时要扩展的知识是：volatile 这个C/C++中的关键字，它请求编译器不缓存这个变量的数据，而是每次都从内存中读取。特别是在多线程下共享放变量，必须使用volatile关键字声明才是保险的。

softPwm，软件实现的PWM

树莓派硬件上支持的PWM输出的引脚有限，为了突破这个限制，wiringPi提供了软件实现的PWM输出API。

需要包含头文件：#include <softPwm.h>

编译时需要添pthread库链接 -lpthread

int softPwmCreate (int pin, int initialValue, int pwmRange)	pin：用来作为软件PWM输出的引脚 initalValue：引脚输出的初始值 pwmRange：PWM值的范围上限 建议使用100. 返回：0表示成功。	使用一个指定的pin引脚创建一个模拟的PWM输出引脚
void softPwmWrite (int pin, int value)	pin：通过softPwmCreate创建的引脚 value：PWM引脚输出的值	更新引脚输出的PWM值

串口通信

使用时需要包含头文件：#include <wiringSerial.h>

int serialOpen (char *device, int baud)	device:串口的地址，在Linux中就是设备所在的目录。 默认一般是"/dev/ttyAMA0",我的是这样的。 baud：波特率 返回：正常返回文件描述符，否则返回-1失败。	打开并初始串口
void serialClose (int fd)	fd：文件描述符	关闭fd关联的串口
void serialPutchar (int fd, unsigned char c)	fd:文件描述符 c:要发送的数据	发送一个字节的数据到串口
void serialPuts (int fd, char *s)	fd：文件描述符 s：发送的字符串，字符串要以'\0'结尾	发送一个字符串到串口
void serialPrintf (int fd, char *message, …)	fd：文件描述符 message：格式化的字符串	像使用C语言中的printf一样发送数据到串口
int serialDataAvail (int fd)	fd：文件描述符 返回：串口缓存中已经接收的，可读取的字节数，-1代表错误	获取串口缓存中可用的字节数。
int serialGetchar (int fd)	fd：文件描述符 返回：读取到的字符	从串口读取一个字节数据返回。 如果串口缓存中没有可用的数据，则会等待10秒，如果10后还有没，返回-1 所以，在读取前，做好通过serialDataAvail判断下。
void serialFlush (int fd)	fd：文件描述符	刷新，清空串口缓冲中的所有可用的数据。
*size_t write (int fd,const void * buf,size_t count)	fd：文件描述符 buf：需要发送的数据缓存数组 count:发送buf中的前count个字节数据 返回：实际写入的字符数，错误返回-1	这个是Linux下的标准IO库函数，需要包含头文件 #include <unistd.h> 当要发送到的数据量过大时，wiringPi建议使用这个函数。
*size_t read(int fd,void * buf ,size_t count);	fd：文件描述符 buf：接受的数据缓存的数组 count:接收的字节数. 返回：实际读取的字符数。	这个是Linux下的标准IO库函数，需要包含头文件 #include <unistd.h> 当要接收的数据量过大时，wiringPi建议使用这个函数。

初次使用树莓派串口编程，需要配置。我开始搞了很久，以为是程序写错了 还一直在调试。。。(~￣▽￣)~

```
/* 修改 cmdline.txt文件 */
>cd /boot/
>sudo vim cmdline.txt


删除【】之间的部分
dwc_otg.lpm_enable=0  【console=ttyAMA0,115200】  kgdboc=ttyAMA0,115200 console=tty1 root=/dev/mmcblk0p2 root


/*修改 inittab文件 */
>cd /etc/
>sudo vim inittab

注释掉最后一行内容:，在前面加上 # 号
#T0:23:respawn:/sbin/getty -L ttyAMA0 115200 vt100


sudo reboot 重启
```

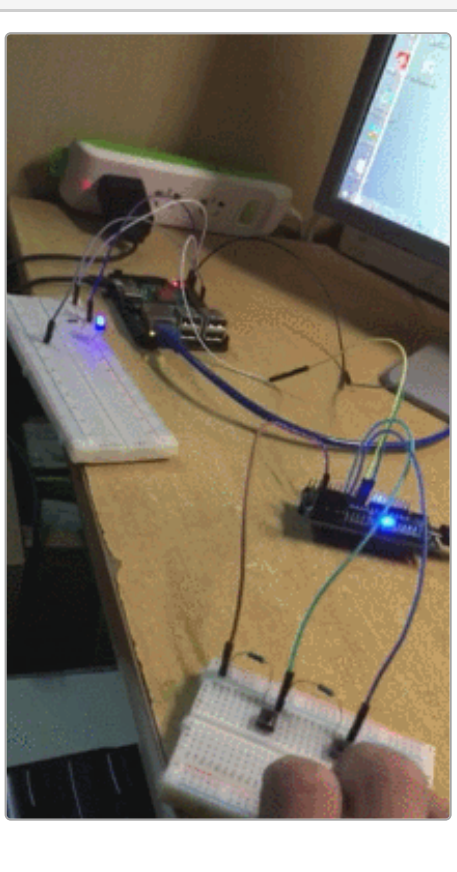
下面是双机通信的一个例子

C51代码，作为串口通信的接发送。serial库请看另一篇文章

View Code

树莓派代码，作为串口通信的接收方

View Code



shift移位寄存器芯片API

需要包含头文件 #include <wiringShift.h>

void shiftOut (uint8_t dPin, uint8_t cPin, uint8_t order, uint8_t val)	dPin：移位芯片的串行数据入口引脚，比如74HC595的SER脚 cPin：移位芯片的时钟引脚。如74HC595的11脚 order： LSBFIRST 先发送数据的低位 MSBFIRST先发送数据的高位 val：要发送的8位数据	将val串化，通过芯片转化为并行输出 如常见的74HC595
uint8_t shiftIn (uint8_t dPin, uint8_t cPin, uint8_t order)	同上。	将并行数据，通过芯片转化为串行输出。

用过595的都知道还有一个引脚：12脚，Rpin，用于把移位寄存器中的数据更新到存储寄存器中，然后wiringPi的API中没有使用这个引脚。我建议使用的时候自己加上。

View Code

树莓派硬件平台特有的API

并没有列全，我只是列出了相对来说有用的，其他的，都基本不会用到。

pwmSetMode (int mode)	mode：PWM运行模式	设置PWM的运行模式。 pwm发生器可以运行在2种模式下，通过参数指定： PWM_MODE_BAL ：树莓派默认的PWM模式 PWM_MODE_MS ：传统的pwm模式，
pwmSetRange (unsigned int range)	range，范围的最大值 0~range	设置pwm发生器的数值范围，默认是1024
pwmSetClock (int divisor)		This sets the divisor for the PWM clock. To understand more about the PWM system, you'll need to read the Broadcom ARM peripherals manual.
piBoardRev (void)	返回：树莓派板子的版本编号 1或者2	/

就这样，以后会更新。

欢迎转载，请注明出处：www.cnblogs.com/lulipro

为了获得更好的阅读体验，请访问原博客地址。

限于本人水平，如果文章和代码有表述不当之处，还请不吝赐教。

代码钢琴家

作者：lulipro–代码钢琴家
出处：http://www.cnblogs.com/lulipro/
本文版权归作者和博客园共有，欢迎转载。限于本人水平，如果文章和代码有表述不当之处，还请不吝赐教。

分类： 树莓派

好文要顶 关注我 收藏该文

lulipro
粉丝 - 356 关注 - 22
+加关注

18 推荐 0 反对
升级成为会员

« 上一篇： C++头文件，预处理详解
» 下一篇： 使用Notepad++代替笨拙的Arduino IDE

posted @ 2016-10-27 21:54 lulipro 阅读(104612) 评论(20) 编辑 收藏 举报

弹尽粮绝，会员救园：会员上线，命悬一线 刷新页面 返回顶部

登录后才能查看或发表评论，立即 登录 或者 逛逛 博客园首页

【推荐】中国亚运历史首金影像感动上映，49年前珍贵瞬间，一起用AI重现
【推荐】阿里云-云服务器省钱攻略 ：五种权益，限时发放，不容错过

阿里云
首金
中国亚运历史首金影像感动上映
49年前珍贵瞬间，一起用AI重现
立即观看>

编辑推荐：
- 理解并掌握 C# 的 Channel：从使用案例到源码解读
- 一次提高接口并发的经历
- 从内核世界透视 mmap 内存映射的本质（原理篇）
- ［.NET8］访问私有成员新姿势 UnsafeAccessor（下）
- [MAUI]实现动态拖拽排序网格

阅读排行：
- C#开源、功能强大、免费的Windows系统优化工具 - Optimizer
- 好用！这些工具国庆一定要研究下「GitHub 热点速览」
- 理解并掌握C#的Channel：从使用案例到源码解读（一）
- .NET开发工作效率提升利器 - CodeGeeX AI编程助手
- 记一次 .NET某新能源MES系统 非托管泄露

公告

昵称： lulipro
园龄： 7年9个月
粉丝： 356
关注： 22
+加关注

随笔分类
- Arduino(9)
- C/C++(4)
- C语言和操作系统(3)
- FLTK(1)
- Linux学习(2)
- 树莓派(5)
- 数据结构(9)
- 刷题记录(3)
- 算法(6)

阅读排行榜
1. 树莓派wiringPi库详解(104609)
2. C语言指针详解(104009)
3. 一次性搞清楚equals和hashCode(62460)
4. 如何编写自己的Arduino库？(50303)
5. Arduino基本数据类型(42370)

评论排行榜
1. C语言指针详解(23)
2. 树莓派wiringPi库详解(20)
3. 一次性搞清楚equals和hashCode(16)
4. 用Arduino剖析PWM脉宽调制(8)
5. 如何编写自己的Arduino库？(8)

推荐排行榜
1. C语言指针详解(65)
2. 树莓派wiringPi库详解(18)
3. 一次性搞清楚equals和hashCode(18)
4. 【算法】表达式求值--逆波兰算法介绍(12)
5. 如何编写自己的Arduino库？(12)

记一次 .NET某新能源MES系统 非托管泄露

Copyright © 2023 lulipro
Powered by .NET 7.0 on Kubernetes